APUNTES

UD03 — Configuración de Linux

Módulo: Sistemas Operativos Monopuesto · 1º SMR

Introducción

En UD02 instalaste tu máquina virtual con Ubuntu, la actualizaste y aprendiste a protegerla con instantáneas antes de hacer cambios arriesgados. Esta unidad no vuelve a instalar nada: te enseña a **vivir con ese sistema el día a día**, que es lo que de verdad va a ocupar la mayor parte del trabajo de un técnico de sistemas — arrancarlo, trabajar en el entorno que prefieras, mantenerlo actualizado, recuperarlo si algo falla y automatizar lo que se repite.

Al terminar esta unidad debes ser capaz de:

- Explicar el proceso de arranque de Linux (GRUB, systemd) y gestionar sesiones de usuario.
- Diferenciar la línea de comandos del entorno gráfico, e instalar, cambiar y personalizar varios entornos de escritorio en el mismo sistema.
- Gestionar sistemas de archivos específicos de Linux: montaje manual y automático mediante `/etc/fstab`.
- Aplicar métodos de recuperación del sistema operativo ante fallos de arranque, incluyendo la reparación de GRUB.
- Mantener el sistema actualizado y gestionar la instalación y desinstalación de software con APT, con nociones comparadas de DNF.
- Usar asistentes de configuración de red y dispositivos.
- Automatizar tareas del sistema con `cron` y `at`.

Cómo está pensada esta unidad: cada vez que aparezca un comando nuevo, vas a encontrar un cuadro **"Vamos a practicar"** con dos partes: primero una guía **paso a paso** con lo que tienes que escribir literalmente en tu terminal, para que compruebes con tus propios ojos qué hace ese comando; después, una tarea algo más abierta que te obliga a razonar, no solo a copiar. Haz siempre la parte guiada antes de pasar a la abierta — es la forma de que el comando se te quede grabado de verdad.

1. Arranque y sesiones

Qué pasa entre que arranca GRUB y ves tu escritorio

En UD02 viste que, tras el firmware (BIOS/UEFI), el gestor de arranque (GRUB en Linux) carga el sistema operativo elegido. Pero "cargar el sistema operativo" no es un único paso: GRUB carga el **núcleo** (*kernel*) de Linux, y el núcleo, a su vez, arranca el primer proceso que se ejecuta en espacio de usuario: **systemd**, con el identificador de proceso (PID) número 1.

systemd es el **sistema de inicio** (*init system*) que usan hoy la inmensa mayoría de distribuciones Linux, incluida Ubuntu. Su trabajo es arrancar el resto de servicios y procesos del sistema, respetando el orden y las dependencias entre ellos: la red no puede arrancar antes de que existan las interfaces de red, un servicio que depende de una base de datos no puede arrancar antes que esa base de datos, y así sucesivamente.

Unidades y targets

systemd organiza todo lo que gestiona en **unidades** (*units*). Una unidad `.service` es un servicio; una unidad `.mount` es un punto de montaje; una unidad `.timer` es una tarea programada (la verás en el epígrafe 6). Un **target** (`.target`) es un conjunto de unidades que definen un "estado" concreto del sistema — el equivalente moderno a los antiguos *runlevels* que usaban los sistemas de inicio anteriores a systemd.

Los dos targets que más te interesan aquí son:

- **`multi-user.target`**: el sistema arranca completo, con red y todos los servicios, pero **sin interfaz gráfica** — solo modo texto.
- **`graphical.target`**: incluye todo lo anterior, más el gestor de pantalla y el entorno de escritorio.

El comando `systemctl` es la herramienta central para consultar y gestionar unidades y targets. `systemctl get-default` muestra el target por defecto; `systemctl set-default multi-user.target` lo cambia de forma permanente; `systemctl isolate multi-user.target` cambia el estado **ahora mismo**, sin tocar la configuración de arranque — es completamente reversible con `systemctl isolate graphical.target`, sin necesidad de reiniciar.

`journalctl`: ver qué ha pasado en el arranque

systemd registra toda la actividad del sistema en un registro centralizado llamado el **journal**. El comando `journalctl -b` muestra los mensajes del arranque actual; `journalctl -b -1`, los del arranque anterior. Es tu primera herramienta de diagnóstico real del curso, y la vas a volver a usar en unidades posteriores para investigar problemas de servicios.

Sesiones de usuario

Una **sesión** es el periodo en el que un usuario autenticado tiene procesos asociados a su cuenta, ya sea en una terminal (TTY) o de forma gráfica. Puede haber **varias sesiones activas a la vez**: por ejemplo, tú conectado por pantalla gráfica y, al mismo tiempo, alguien conectado por SSH desde otro equipo, o dos usuarios distintos en TTYs diferentes. `loginctl list-sessions` muestra todas las sesiones activas del sistema; `loginctl show-session <ID>` da el detalle de una en concreto.

Cerrar sesión, bloquear, apagar y reiniciar no son lo mismo

Es un error muy habitual mezclar estos cuatro conceptos:

- **Cerrar sesión** (*logout*): vuelve a la pantalla de inicio de sesión. El sistema sigue completamente encendido.

- **Bloquear**: la sesión sigue activa exactamente donde la dejaste, solo que pide contraseña para volver a verla.
- **Apagar** (`systemctl poweroff` o `shutdown -h now`): el equipo se apaga por completo.
- **Reiniciar** (`systemctl reboot`): el equipo se apaga y vuelve a arrancar.

VAMOS A PRACTICAR: CAMBIA DE TARGET Y CONSULTA EL REGISTRO DE ARRANQUE

Paso a paso, en tu terminal:

1. `systemctl get-default` — anota el resultado (debería ser `graphical.target`).
2. `sudo systemctl isolate multi-user.target` — la pantalla debería quedarse en modo texto.
3. Inicia sesión con tu usuario y contraseña en esa terminal de texto, y ejecuta `whoami` para comprobar que sigues siendo el mismo usuario que en el escritorio.
4. `sudo systemctl isolate graphical.target` — recuperas el entorno gráfico, sin haber reiniciado en ningún momento.
5. `journalctl -b | less` — despliega el registro del arranque actual (usa las flechas para moverte, `q` para salir) y localiza alguna línea relacionada con el montaje del sistema de archivos raíz.
6. `loginctl list-sessions` — anota el ID de tu sesión.
7. `loginctl show-session <ID>` (sustituyendo `<ID>` por el que acabas de anotar) — observa la información detallada que te da: usuario, tipo de sesión, estado.

Reflexiona: explica, en tus propias palabras, qué diferencia real hay a nivel de systemd entre `multi-user.target` y `graphical.target`. No te quedes en "uno tiene interfaz gráfica y el otro no": explica la relación de dependencia entre ambos targets.

2. Interfaces de usuario y entornos de escritorio

Línea de comandos y entorno gráfico: cuándo conviene cada una

Ya viste en UD01 que el shell puede ser en texto (línea de comandos) o en gráficos. Ninguna sustituye por completo a la otra: la **línea de comandos** (CLI, *Command Line Interface*) es más rápida para tareas repetitivas o automatizables, funciona igual en local que conectado por SSH a un equipo remoto, y consume muy pocos recursos. El **entorno gráfico** (GUI, *Graphical User Interface*) tiene una curva de aprendizaje menor y resulta más intuitivo para tareas exploratorias. Un técnico competente se mueve entre ambas según lo que la tarea concreta le pida, no por preferencia fija.

Qué es exactamente un entorno de escritorio

Un **entorno de escritorio** (*Desktop Environment*, DE) es el conjunto de componentes gráficos que conforman la experiencia visual e interactiva de un usuario: el gestor de ventanas, el panel o barra de tareas, el gestor de archivos gráfico, las notificaciones, el tema visual. No es lo mismo que "el shell gráfico" en abstracto que viste en UD01 — aquí hablamos de implementaciones concretas, y distintas entre sí, que puedes instalar y comparar.

Los más habituales que te vas a encontrar en el sector son:

- **GNOME**: el que trae Ubuntu por defecto. Basado en GTK, con un diseño minimalista pensado para pantalla completa y gestos.
- **KDE Plasma**: basado en Qt, muy personalizable, con un aspecto más tradicional de escritorio y barra de tareas.
- **XFCE**: ligero, pensado para hardware limitado, visualmente sencillo.
- **MATE**: continuación del clásico GNOME 2, con un escritorio de estilo tradicional.

Ninguno es "mejor" en abstracto — la elección depende de gustos, del hardware disponible y del caso de uso, exactamente igual que viste con los tipos de núcleo del sistema operativo en UD01.

El gestor de pantalla y la elección de sesión

El **gestor de pantalla** (*display manager*) es el que muestra la pantalla de inicio de sesión gráfica y arranca el entorno de escritorio correspondiente. GDM está asociado tradicionalmente a GNOME, SDDM a KDE Plasma, y LightDM es una opción ligera pensada para funcionar bien con cualquier DE.

Aquí está la parte que más sorprende la primera vez: **puedes tener varios entornos de escritorio instalados a la vez en el mismo sistema**, sin que uno sustituya al otro. Cuando hay más de uno instalado, la propia pantalla de inicio de sesión ofrece un icono o un menú para elegir con cuál quieres entrar esa vez — sin necesidad de desinstalar nada.

Instalar un segundo entorno se hace igual que instalar cualquier otro software, con el gestor de paquetes que verás formalizado en el epígrafe 4: por ejemplo, `sudo apt install xubuntu-desktop` instala XFCE completo, con su conjunto de aplicaciones por defecto (el paquete `xfce4`, más minimalista, instala solo el núcleo del entorno sin esas aplicaciones añadidas).

VAMOS A PRACTICAR: INSTALA Y ELIGE UN SEGUNDO ENTORNO DE ESCRITORIO

Paso a paso, en tu terminal:

1. `sudo apt update` — deja el índice de paquetes al día antes de instalar nada (lo verás en detalle en el apartado 4).
2. `sudo apt install xubuntu-desktop` (o el meta-paquete que se te indique: `kubuntu-desktop` para KDE Plasma, `ubuntu-mate-desktop` para MATE) — tardará varios minutos, es una instalación grande con muchos paquetes.
3. Cuando termine, cierra la sesión desde el menú del escritorio (**no apagues la VM**).
4. En la pantalla de inicio de sesión, busca el icono de engranaje junto al campo de contraseña, y comprueba que aparecen ya las dos sesiones disponibles.
5. Entra con el nuevo entorno de escritorio.

Ejemplo resuelto: tras `sudo apt install xubuntu-desktop` y reiniciar la sesión, en la pantalla de login aparece un icono de engranaje junto al campo de contraseña; al pulsarlo, se despliega la lista de sesiones disponibles (GNOME y XFCE), y eliges la que quieras usar en ese inicio de sesión concreto.

VAMOS A PRACTICAR: PERSONALIZA Y COMPARA

Paso a paso:

1. En tu entorno original (por ejemplo, GNOME), abre "Ajustes" y cambia el fondo de pantalla y el tema.
2. Cierra sesión, entra con el segundo entorno, y localiza su panel de configuración equivalente (por ejemplo, "Configuración del sistema" en KDE Plasma, o el "Gestor de configuración" en XFCE).
3. Cambia también ahí el fondo de pantalla y el tema.
4. Abre el monitor de recursos (o el gestor de tareas gráfico) en cada sesión, y anota cuánta RAM usa el sistema recién iniciado en cada una.
5. Completa una tabla con: RAM en reposo, primera impresión visual, y dónde está la configuración de personalización en cada entorno.

Reflexiona: si desinstalaras ahora mismo el entorno que no estás usando, ¿qué riesgo real correrías? ¿Por qué es más seguro simplemente no usarlo que desinstalarlo?

3. Sistemas de archivos y recuperación

Montar un sistema de archivos, paso a paso

"Montar" un dispositivo o partición significa asociarlo a un punto concreto del árbol de directorios, para poder acceder a su contenido a través de esa ruta. `mount` monta manualmente; `umount` desmonta. Los puntos de montaje típicos para dispositivos externos son `/mnt` y `/media`.

No necesitas un disco físico nuevo para practicar esto: puedes crear un archivo que **actúe como si fuera un disco** (una técnica real y muy habitual para pruebas, llamada montaje *loopback*), formatearlo con un sistema de archivos y montarlo exactamente igual que montarías un disco de verdad.

VAMOS A PRACTICAR: MONTA UN "DISCO" DE PRUEBA A MANO

Paso a paso, en tu terminal:

1. `sudo dd if=/dev/zero of=/root/disco_prueba.img bs=1M count=100` — crea un archivo de 100 MB que va a hacer de disco. Hace falta `sudo` porque lo guardamos en `/root`, la carpeta personal del administrador, en la que tu usuario no puede escribir.
2. `sudo mkfs.ext4 /root/disco_prueba.img` — lo formatea con el sistema de archivos ext4 (el mismo que usa tu VM).
3. `sudo mkdir /mnt/prueba` — crea el punto de montaje.
4. `sudo mount -o loop /root/disco_prueba.img /mnt/prueba` — lo monta manualmente, como si fuera un disco real.
5. `df -h | grep prueba` — comprueba que aparece montado, con su tamaño y espacio disponible.
6. `echo "hola" | sudo tee /mnt/prueba/saludo.txt` — crea un archivo dentro.
7. `sudo umount /mnt/prueba` — desmóntalo.
8. `ls /mnt/prueba` — comprueba que la carpeta aparece vacía.

Reflexiona: el archivo `saludo.txt` no se ha borrado — sigue existiendo dentro de `disco_prueba.img`. Entonces, ¿por qué "desaparece" de `/mnt/prueba` en cuanto desmontas? ¿Qué te dice esto sobre lo que significa realmente "montar" algo?

`/etc/fstab`: qué se monta solo en cada arranque

Montar a mano cada vez que arrancas sería muy poco práctico para los discos que usas siempre. El archivo **`/etc/fstab`** resuelve esto: define qué se monta automáticamente en cada arranque, sin intervención manual. Cada línea sigue esta sintaxis:

```
<dispositivo>  <punto de montaje>  <tipo de sistema de archivos>  <opciones>  <dump>
<pass>
```

- **Dispositivo:** qué se monta (luego verás cómo identificarlo).
- **Punto de montaje:** en qué carpeta aparecerá su contenido.
- **Tipo:** el sistema de archivos (`ext4`, `vfat`, `ntfs`, `swap`...).
- **Opciones:** cómo se monta. `defaults` aplica las opciones habituales; hay otras, como `nofail` (arrancar aunque el dispositivo no esté) o `ro` (solo lectura).
- **Dump:** una opción heredada de una antigua herramienta de copias de seguridad. Hoy se deja casi siempre a `0`.
- **Pass:** el orden en que se comprueba el sistema de archivos al arrancar: `1` para la raíz, `2` para el resto y `0` para no comprobarlo.

Se recomienda identificar el dispositivo por su **UUID** (un identificador único y estable para cada sistema de archivos) en vez de por su nombre de dispositivo. La razón es puramente práctica: un nombre como `/dev/sdb1` puede cambiar entre arranques si se añaden o quitan discos, mientras que el UUID no cambia nunca. El comando `blkid` muestra el UUID de cada partición.

Hay un detalle más que conviene conocer: systemd no lee `fstab` cada vez que lo necesita. Lo lee al arrancar y convierte cada línea en una **unidad `.mount`**, del mismo tipo que las que viste en el apartado 1. Por eso, cuando editas `fstab` con el sistema en marcha, tienes que pedirle a systemd que lo vuelva a leer con `sudo systemctl daemon-reload`.

¿Y el disco de prueba que acabas de montar a mano? Para `fstab` no sirve. `blkid` te mostrará un UUID si se lo pides sobre el archivo `disco_prueba.img`, pero al arrancar, systemd y `mount` buscan ese UUID entre los **dispositivos reales** del sistema, y un archivo de imagen no lo es. Resultado: el disco "no aparece nunca" y el arranque se detiene esperándolo. Para practicar `fstab` de verdad, vas a añadir a tu VM un segundo disco, exactamente como harías con un disco nuevo en un equipo real.

VAMOS A PRACTICAR: AÑADE UN SEGUNDO DISCO Y HAZ QUE SE MONTE SOLO

Paso a paso:

1. **Con la VM apagada**, en VirtualBox abre **Configuración** → **Almacenamiento**, selecciona el controlador SATA y pulsa el icono de **añadir disco duro**. Elige **Crear**, tipo VDI, reservado dinámicamente, de **1 GB**, con el nombre `datos`. Selecciónalo y acepta.
2. Arranca la VM y ejecuta `lsblk`. Deberías ver un disco nuevo de 1G sin particiones, normalmente `sdb` (tu disco del sistema es `sda`). Anota su nombre exacto.
3. `sudo parted /dev/sdb --script mklabel gpt mkpart datos ext4 0% 100%` — crea en el disco nuevo una tabla de particiones GPT y una partición que lo ocupa entero. **Comprueba antes que `sdb` es el disco nuevo, el de 1G: si pones el disco de tu sistema, borrarás su tabla de particiones.**
4. `lsblk` de nuevo — ahora aparece `sdb1`, la partición que acabas de crear.
5. `sudo mkfs.ext4 /dev/sdb1` — le da formato ext4.
6. `sudo mkdir /mnt/datos` — crea el punto de montaje.
7. `sudo blkid /dev/sdb1` — anota el UUID de la **partición** (`sdb1`, no `sdb`).
8. `sudo nano /etc/fstab` y añade al final la línea `UUID=<tu-uuid> /mnt/datos ext4 defaults 0 2`, sustituyendo `<tu-uuid>` por el UUID real que has anotado.
9. `sudo systemctl daemon-reload` — systemd vuelve a leer `fstab`.
10. `sudo mount -a` — monta todo lo que indica `fstab` y no está montado todavía. Si hay un error en tu línea, te lo dirá ahora, con el sistema en marcha, y no en el próximo arranque. Comprueba con `df -h /mnt/datos` que el disco está montado.
11. Reinicia la VM y, tras el arranque, comprueba de nuevo con `df -h /mnt/datos` que sigue montado sin que hayas hecho nada.

Ejemplo resuelto: `blkid` devuelve `/dev/sdb1: UUID="1a2b3c4d-5e6f-..." TYPE="ext4" PARTLABEL="datos"`. La línea añadida a `fstab` queda `UUID=1a2b3c4d-5e6f-... /mnt/datos ext4 defaults 0 2`. Tras `sudo systemctl daemon-reload` y `sudo mount -a`, `df -h` muestra `/dev/sdb1` montado en `/mnt/datos`, con algo menos de 1 GB de tamaño.

Reflexiona: ¿por qué te pide el paso 10 ejecutar `mount -a` antes de reiniciar? ¿Qué ganas probándolo así?

La excepción a la regla del UUID: el swap

En UD01 viste que Linux usa **swap** para la paginación por demanda, cuando la RAM no basta. Si echas un vistazo a tu propio `/etc/fstab`, vas a encontrar una línea para el swap que **no sigue la recomendación de usar UUID** — algo así:

```
/swapfile none swap sw 0 0
```

La razón es que, desde hace varias versiones, Ubuntu ya no crea una **partición** de swap dedicada: crea un **archivo**, `/swapfile`, dentro del propio sistema de archivos raíz — exactamente el mismo mecanismo que el `pagefile.sys` de Windows que se mencionó en UD01, solo que con otro nombre. Como no es una partición identificable con `blkid`, sino un archivo más dentro de `/`, en `fstab` se indica por su ruta directamente, sin UUID.

¿Por qué el cambio? Tres motivos prácticos: un particionado más simple (cambiar el tamaño del swap ya no exige tocar el esquema de particiones, basta con recrear el archivo), mejor integración con el cifrado de disco (un archivo de swap dentro de una partición ya cifrada queda cifrado automáticamente, sin gestión aparte) y porque la diferencia de rendimiento que antes hacía preferible una partición dedicada ha desaparecido casi por completo con los kernels actuales. Sigue siendo posible crear una partición de swap tradicional si se prefiere — ya no es la opción por defecto, pero no ha desaparecido como posibilidad.

VAMOS A PRACTICAR: LOCALIZA TU PROPIO SWAP

`swapon --show` y `free -h` — comprueba cuánto swap tiene tu sistema y de dónde procede. Después, busca la línea correspondiente en tu `/etc/fstab` con `cat /etc/fstab`.

Reflexiona: ¿por qué esta línea concreta no representa ningún riesgo de los que hemos visto con el UUID (el problema de que `/dev/sdaX` pueda cambiar de nombre entre arranques)?

Cuando el arranque falla: no es el fin del mundo

Un error en `/etc/fstab` puede hacer que el sistema no complete su arranque con normalidad. Si una línea indica un disco que no aparece, o un montaje que no se puede hacer, y no lleva la opción `nofail`, systemd espera un rato al dispositivo (hasta un minuto y medio) y después se detiene en el **modo de emergencia** (`emergency.target`). Es importante que interiorices esto: **no se ha perdido ningún dato**. El sistema simplemente no puede continuar hasta que se corrija la configuración que le impide montar todo lo que tiene indicado.

Aviso importante, relacionado con lo visto en UD01 sobre journaling: el journaling de ext4 protege la consistencia del sistema de archivos ante cortes de luz o interrupciones bruscas, pero **no protege de un error humano de configuración** como un `fstab` mal escrito. Son dos problemas de naturaleza completamente distinta, y confundirlos es un error conceptual habitual.

Entrar a reparar cuando Ubuntu no te deja

En muchas distribuciones, el modo de emergencia pide la contraseña de root y te da una terminal para corregir el error. **En Ubuntu, la cuenta de root está bloqueada por defecto**: tu usuario administra el sistema con `sudo`, pero root no tiene contraseña con la que entrar. Por eso, lo normal es que el modo de emergencia muestre un mensaje como `Cannot open access to console, the root account is locked` y no te deje hacer nada.

La solución es entrar "por otra puerta": pedirle a GRUB que, en lugar de arrancar el sistema normal, arranque directamente una terminal de root. Para ello, se edita la línea de arranque del núcleo y se le añade el parámetro `init=/bin/bash`. Recuerda del apartado 1 que el primer

proceso que arranca el núcleo es systemd; con `init=/bin/bash` le dices que, en su lugar, arranque una terminal. No se ejecuta nada más: ni systemd, ni `fstab`, ni el escritorio. Solo la terminal, con el disco del sistema montado en modo **solo lectura**, así que lo primero es volver a montarlo en modo escritura.

Otra vía, más lenta pero que funciona siempre, es arrancar desde una **ISO de Ubuntu en modo *live*** (la que descargaste en UD02), montar desde ahí el disco del sistema y corregir el archivo.

Que cualquiera con acceso al menú de GRUB pueda hacer esto tiene una lectura de seguridad importante: **quien tiene acceso físico a un equipo, tiene el equipo**. Por eso en entornos profesionales se protege GRUB con contraseña y se cifra el disco, algo que verás en el módulo de Seguridad Informática.

Reparar GRUB cuando el propio gestor de arranque falla

Si el problema no es `fstab` sino el propio gestor de arranque (una situación que ya viste en UD02 con el dual boot), el procedimiento estándar es más laborioso: arrancar desde un live USB, montar la partición raíz del sistema instalado, montar también dentro `/dev`, `/proc` y `/sys`, hacer `chroot` a ese sistema montado (una forma de "entrar" en él como si fuera el sistema en marcha) y ejecutar `grub-install` y `update-grub` desde dentro. Tu profesor o profesora te lo mostrará en directo, ya que es un procedimiento con muchos pasos.

VAMOS A PRACTICAR: ROMPE Y REPARA FSTAB, DE FORMA CONTROLADA

Paso a paso:

1. Toma una instantánea de tu VM con el nombre "Antes de romper fstab" — es exactamente el tipo de cambio arriesgado para el que aprendiste a usar snapshots en UD02.
2. `sudo nano /etc/fstab` y, en la línea de `/mnt/datos` que añadiste antes, cambia un par de caracteres del UUID para que no corresponda a ningún disco.
3. Reinicia la VM. El arranque parecerá bloqueado durante un minuto y medio: systemd está esperando a un disco con ese UUID, que no va a aparecer nunca.
4. Lee con atención el mensaje que aparece y anótalo literalmente, junto con la unidad o el dispositivo que señala como responsable del fallo.
5. Si el mensaje dice que la cuenta de root está bloqueada, no insistas: reinicia la VM desde el menú de VirtualBox (**Máquina** → **Reiniciar**).
6. Justo al empezar el arranque, mantén pulsada la tecla `Mayús` (si tu VM arranca en modo BIOS) o pulsa `Esc` varias veces (si arranca en modo UEFI) hasta que aparezca el menú de GRUB.
7. Con la entrada de Ubuntu seleccionada, pulsa `e`. Busca la línea que empieza por `linux`, ve al final de esa línea y añade un espacio y `init=/bin/bash`. Arranca con `Ctrl+X` o `F10`.
8. `mount -o remount,rw /` — vuelve a montar el disco del sistema en modo escritura.
9. `blkid /dev/sdb1` — consulta el UUID correcto. Después, `nano /etc/fstab` y corrige la línea.
10. `sync` (asegura que el cambio se escribe en el disco) y después `reboot -f`.
11. Comprueba que el sistema arranca con normalidad y que `df -h /mnt/datos` vuelve a mostrar el disco montado.

Ojo con el teclado: en el editor de GRUB, y puede que también en la terminal que arranca con `init=/bin/bash`, el teclado funciona con la distribución estadounidense, no con la española. Con un teclado español, la `/` está en la tecla `-`, el `=` en la tecla `¡` y el `-` en la tecla `'`.

Documenta con capturas el mensaje de error, el proceso de diagnóstico y corrección, y la comprobación final.

Reflexiona: explica en 3-4 líneas por qué esta incidencia no tiene nada que ver con el journaling de ext4 que estudiaste en UD01. Y después: si hubieras escrito la línea con `defaults,nofail` en lugar de `defaults`, ¿qué habría pasado en el paso 3?

4. Gestión de software: actualización e instalación

`apt update` y `apt upgrade` no son lo mismo

Es, con diferencia, la confusión más habitual de este bloque, así que conviene fijarla bien desde el principio: **`apt update` no instala nada**. Su única función es refrescar el índice local de paquetes disponibles, consultando los repositorios configurados — es como actualizar el catálogo de una tienda para saber qué hay disponible. **`apt upgrade`** es el que de verdad instala las actualizaciones de los paquetes ya instalados en tu sistema, usando ese índice ya refrescado. Por eso casi siempre se ejecutan juntos y en ese orden: `sudo apt update && sudo apt upgrade`.

`apt full-upgrade` (también llamado `dist-upgrade`) hace lo mismo que `upgrade`, pero permitiendo además instalar o eliminar paquetes si hace falta para resolver dependencias mayores entre versiones.

VAMOS A PRACTICAR: ACTUALIZA EL ÍNDICE E INSTALA TU PRIMER PAQUETE

Paso a paso, en tu terminal:

1. `sudo apt update` — refresca el índice de paquetes disponibles (todavía no instala nada).
2. `sudo apt install tree` — instala una pequeña utilidad que dibuja la estructura de carpetas en forma de árbol.
3. `tree ~` — pruébala sobre tu carpeta personal: te va a recordar mucho a los árboles de directorios que dibujamos a mano en UD01.
4. `apt show tree` — consulta la información del paquete: versión, tamaño, descripción, de dónde viene.

De dónde vienen los paquetes: repositorios y PPA

APT consulta una lista de **repositorios** configurada en `/etc/apt/sources.list` (y en los archivos dentro de `/etc/apt/sources.list.d/`) para saber dónde buscar el software disponible. Un **PPA** (*Personal Package Archive*) es un repositorio adicional, normalmente mantenido por terceros, que añade software que no está en los repositorios oficiales de Ubuntu.

Añadir un PPA equivale a confiar en quien lo mantiene — exactamente el mismo criterio de seguridad que ya viste con el origen de las ISOs en UD02: solo se deben añadir repositorios de fuentes conocidas y fiables, nunca por pura comodidad de "que funcione ya".

Instalar y desinstalar: la diferencia entre `remove` y `purge`

- `apt install <paquete>` instala el paquete indicado.
- `apt remove <paquete>` lo desinstala, pero **conserva sus archivos de configuración**, por si decides reinstalarlo más adelante.
- `apt purge <paquete>` lo desinstala y **también elimina esos archivos de configuración**.

- `apt autoremove` limpia paquetes que quedaron instalados solo como dependencia de otro, y que ya no necesita ningún paquete instalado.

Snap: otro gestor de paquetes en Ubuntu

Además de APT, Ubuntu incluye **Snap**, el gestor de paquetes propio de Canonical. La diferencia clave es que un paquete Snap va **autocontenido**: empaqueta sus propias dependencias en vez de compartir las del sistema, así que el mismo `.snap` funciona igual en distintas versiones de Ubuntu — a cambio de ocupar más espacio en disco y de un cierto aislamiento (*sandboxing*) respecto al resto del sistema, que limita el acceso a partes sensibles como el propio gestor de arranque.

`snap install <paquete>` instala; `snap list` muestra lo que tienes instalado, con su versión y canal de actualización; `snap remove <paquete>` desinstala; `snap remove --purge <paquete>` además borra los datos guardados de la aplicación — el mismo paralelismo que ya viste entre `apt remove` y `apt purge`. **APT y Snap conviven sin problema en el mismo sistema**: no hay que elegir uno u otro.

VAMOS A PRACTICAR: INSTALA APLICACIONES REALES CON APT Y CON SNAP

Paso a paso, en tu terminal:

1. `sudo apt install refind` — instala rEFInd, un gestor de arranque gráfico alternativo, directamente desde el repositorio oficial de Ubuntu.
2. `dpkg -l | grep refind` — comprueba que aparece instalado ("ii").
3. `sudo add-apt-repository ppa:danielrichter2007/grub-customizer` y `sudo apt update` — añade un PPA (repositorio de terceros) que ofrece GRUB Customizer, una herramienta gráfica para editar el menú de GRUB.
4. `sudo apt install grub-customizer` — instálalo. **Si el PPA no tiene paquetes disponibles para tu versión de Ubuntu, apt te lo dirá con un error** — documenta ese resultado tal cual, es un ejemplo real de por qué conviene comprobar la fiabilidad de un PPA antes de depender de él, y sigue con el resto de la práctica.
5. `sudo snap install vlc` — instala VLC, un reproductor multimedia, esta vez a través de Snap.
6. `snap list vlc` — comprueba que aparece instalado, con su versión y canal.
7. `sudo apt remove refind` seguido de `dpkg -l | grep refind` (verás "rc", configuración residual) y después `sudo apt purge refind` (comprueba que ya no queda ninguna traza).
8. `sudo snap remove --purge vlc` — desinstala VLC por completo.

Reflexiona: ¿por qué crees que una herramienta como GRUB Customizer o rEFInd —que necesitan tocar directamente el gestor de arranque del sistema— casi nunca se distribuyen como Snap?

Instalar un `.deb` suelto

Cuando descargas un archivo `.deb` manualmente, sin pasar por un repositorio, se instala con `sudo dpkg -i paquete.deb`. Si ese paquete necesita otros paquetes que no tienes instalados, `dpkg` falla y deja el sistema con "dependencias rotas" — se soluciona con `sudo apt --fix-broken install`, que hace que APT busque y resuelva automáticamente esas dependencias en los repositorios configurados.

VAMOS A PRACTICAR: PROVOCA (Y ARREGLA) UNAS DEPENDENCIAS ROTAS

Paso a paso:

1. Copia a tu VM el archivo `paquete-practica.deb` que te facilitará tu profesor o profesora (por ejemplo, a través de la carpeta compartida configurada en UD02).
2. `sudo dpkg -i paquete-practica.deb` — la instalación va a fallar, indicando que falta una dependencia. Lee el mensaje con atención: ¿qué paquete dice que necesita?
3. `sudo apt --fix-broken install` — deja que APT localice e instale esa dependencia desde los repositorios, y termine de configurar el paquete pendiente.
4. `dpkg -l | grep paquete-practica` — comprueba que ahora aparece correctamente instalado ("ii", no "iF" ni "rc").
5. Prueba la dependencia que se ha instalado, `cowsay "¡Ya funciona!"`, y el comando que instala el propio paquete: `saludo-smr`.

Reflexiona: ¿por qué `dpkg -i` por sí solo no ha sido capaz de resolver la dependencia que faltaba, mientras que `apt` sí lo consigue automáticamente?

DNF y `.rpm`: el mismo problema, otra familia de distribuciones

En la familia de distribuciones Fedora/RHEL, el gestor de paquetes equivalente a APT es **DNF**, que trabaja con paquetes `.rpm` en vez de `.deb`. No vas a practicar con DNF en el aula porque no tienes una VM de esa familia instalada, pero conviene que conozcas la equivalencia, para no quedarte bloqueado si en el futuro te encuentras con una distribución de este tipo:

APT (Debian/Ubuntu)	DNF (Fedora/RHEL)
`apt update`	`dnf check-update`
`apt upgrade`	`dnf update`
`apt install`	`dnf install`
`dpkg -i paquete.deb`	`rpm -i paquete.rpm`

5. Asistentes de configuración: red y dispositivos

Qué es un asistente de configuración, y por qué existe

Editar archivos de configuración a mano —como acabas de hacer con `/etc/fstab`— es potente, pero propenso a errores. Un **asistente de configuración** es una herramienta, gráfica o de texto interactivo, que guía paso a paso una configuración compleja ofreciendo solo opciones válidas y comprobando automáticamente lo que introduces, reduciendo así el margen de error a cambio de algo menos de control fino.

Red: NetworkManager, desde el entorno gráfico y desde la terminal

NetworkManager es el servicio que gestiona de forma centralizada las conexiones de red en Ubuntu. Puedes acceder a él de tres formas: desde el applet gráfico del escritorio (Configuración → Red), desde `nmtui` (una interfaz de texto interactiva, tipo menú, cómoda para una configuración puntual) o desde `nmcli` (línea de comandos pura, pensada para usarse dentro de scripts y tareas automatizadas — la volverás a ver relacionada con el epígrafe 6).

Netplan y NetworkManager: quién manda realmente

Es una pregunta muy razonable en cuanto oyes hablar de Netplan: "entonces, ¿la IP estática se configura editando el archivo de Netplan?". En un **Ubuntu de escritorio** —el caso de tu VM— la respuesta es **no, no directamente**. El archivo de Netplan (`/etc/netplan/*.yaml`) suele contener una sola línea relevante, `renderer: NetworkManager`, que le dice a Netplan "no gestiones tú las interfaces, delega en NetworkManager". La configuración real de cada conexión vive en NetworkManager (concretamente en `/etc/NetworkManager/system-connections/`), y es a NetworkManager a quien hablan tanto el asistente gráfico como `nmcli`/`nmtui`.

En un **Ubuntu Server** la situación es distinta: ahí sí es habitual editar directamente el `.yaml` de Netplan y aplicar con `sudo netplan apply`, porque normalmente delega en `systemd-networkd` en vez de en NetworkManager. "Netplan" no es una única forma de hacer las cosas: es una capa que puede apoyarse en herramientas distintas según el tipo de instalación.

VAMOS A PRACTICAR: CONFIGURA UNA IP ESTÁTICA, PRIMERO DESDE EL ESCRITORIO

Paso a paso:

1. Abre **Configuración** → **Red** (o Wifi, según tu conexión), pulsa el icono de engranaje de tu conexión y entra en la pestaña **IPv4**.
2. Cambia el método de "Automático (DHCP)" a "Manual", e introduce una dirección IP, máscara y puerta de enlace dentro del rango que permite el modo de red de tu VM (visto en UD02). Aplica los cambios.
3. `ip addr show` — comprueba en la terminal que la IP nueva se ha aplicado.
4. Desde tu **VM Windows 11** (la que instalaste en UD02), abre una terminal (`cmd`) y ejecuta `ping <la IP que acabas de fijar>`. Documenta con una captura que responde correctamente.
5. Vuelve a **Configuración** → **Red** → **IPv4** y cambia el método de nuevo a "Automático (DHCP)".

Reflexiona: para que el `ping` del paso 4 funcione, las dos VMs (Linux y Windows) tienen que "verse" en red. ¿Qué modo de red de VirtualBox (de los que viste en UD02: NAT, Red NAT, Adaptador puente, Solo anfitrión, Red interna) es imprescindible para esto, y por qué el NAT simple no serviría?

VAMOS A PRACTICAR: LA MISMA IP ESTÁTICA, AHORA DESDE LA TERMINAL

Paso a paso:

1. `nmcli connection show` — anota el nombre exacto de tu conexión.
2. `nmcli connection modify "<nombre-de-tu-conexión>" ipv4.method manual ipv4.addresses "<IP>/<prefijo>" ipv4.gateway "<puerta-de-enlace>" ipv4.dns "8.8.8.8"` — fija la misma IP que usaste desde el escritorio (sustituye cada valor por el tuyo).
3. `nmcli connection up "<nombre-de-tu-conexión>"` — aplica el cambio.
4. `ip addr show` — comprueba que coincide con la IP que fijaste antes a mano.
5. Repite el `ping` desde tu VM Windows 11 y comprueba que responde igual que la primera vez.
6. `cat /etc/netplan/*.yaml` — localiza la línea que indica quién gestiona realmente tu red.

Reflexiona: ¿por qué el archivo de Netplan que acabas de ver no contiene tu IP estática, si es supuestamente el sistema de configuración de red de Ubuntu? ¿Dónde está realmente guardada esa configuración?

Dispositivos: impresoras y Bluetooth

El gestor de impresoras de Ubuntu, basado en **CUPS** (*Common UNIX Printing System*), permite añadir una impresora local o de red mediante un asistente gráfico, sin tener que configurar CUPS a mano. El applet de Bluetooth funciona de forma parecida para emparejar dispositivos.

VAMOS A PRACTICAR: EXPLORA EL ASISTENTE DE IMPRESORAS

Abre el gestor de impresoras de tu escritorio (búscalo como "Impresoras" en el menú de aplicaciones) e inicia el asistente para añadir una nueva, aunque no dispongas de ninguna impresora real en el aula. Documenta con capturas cada pantalla del asistente hasta el punto en el que se detiene por falta de una impresora detectada.

Reflexiona: ¿qué información te pide el asistente antes incluso de buscar una impresora? ¿En qué se parece este proceso al de `nmtui` que acabas de usar para la red?

6. Automatización de tareas

`cron`: tareas que se repiten solas

cron es el demonio que ejecuta tareas programadas de forma repetitiva. Cada usuario tiene su propio **crontab**, que se edita con `crontab -e` y se consulta con `crontab -l`. Cada línea sigue esta sintaxis:

```
minuto  hora  día-del-mes  mes  día-de-la-semana  comando
```

Por ejemplo, `0 2 * * *` significa "todos los días a las 2:00 de la madrugada" (el asterisco significa "cualquier valor" en esa posición).

La trampa más habitual: rutas relativas

Cuando escribes un script para automatizarlo con `cron`, la precaución más importante es usar siempre **rutas absolutas** dentro de él (por ejemplo, `/home/tu_usuario/carpeta` en vez de simplemente `carpeta`). El motivo es que el entorno en el que se ejecuta una tarea de `cron` **no es el mismo** que el de una sesión interactiva de terminal: no hereda automáticamente el mismo directorio de trabajo ni las mismas variables de entorno. Un script con rutas relativas que funciona perfectamente al ejecutarlo a mano puede fallar en silencio cuando lo lanza `cron` — es, con diferencia, la causa más habitual de que "algo que funcionaba deje de funcionar en automático".

Otra buena práctica: redirigir la salida del script a un archivo de registro (`>> archivo.log 2>&1`), para poder comprobar después si la tarea se ejecutó correctamente o falló, y por qué.

VAMOS A PRACTICAR: TU PRIMERA TAREA PROGRAMADA CON CRON

Paso a paso, en tu terminal:

1. `nano registro.sh` y escribe dentro:

```
#!/bin/bash
date >> /home/tu_usuario/registro.log
```

(sustituye `tu_usuario` por tu nombre de usuario real, y usa siempre esa ruta absoluta).

2. `chmod +x registro.sh` — dale permisos de ejecución.
3. `./registro.sh` — pruébalo a mano, y comprueba con `cat registro.log` que ha añadido la fecha y hora actuales.
4. `crontab -e` y añade la línea `* * * * * /home/tu_usuario/registro.sh` (ruta absoluta, no `./registro.sh`).
5. Espera 2-3 minutos y ejecuta `cat /home/tu_usuario/registro.log` — deberías ver varias líneas nuevas, una por minuto.
6. `crontab -e` de nuevo y elimina (o comenta con `#` delante) esa línea, para que no siga ejecutándose sin necesidad.

Más allá de "todos los días": otros patrones de temporalidad

El campo de día-de-la-semana admite rangos: `1-5` significa "de lunes a viernes" (el 0 y el 7 son ambos domingo), así que `0 9 * * 1-5` es "todos los días laborables a las 9:00".

¿Y si quieres algo que no encaja directamente en la sintaxis de `cron`, como "en semanas alternas"? No existe un campo para eso — la solución habitual es programar la tarea con la periodicidad más fina que sí soporta `cron` (por ejemplo, cada domingo) y añadir, dentro de la misma línea, una comprobación de la semana ISO del año (`date +%V`) que descarte la mitad de las ejecuciones:

```
0 3 * * 0 [ $(( $(date +\%V) \% 2 )) -eq 0 ] && /home/tu_usuario/backup_personal.sh
```

Aviso de sintaxis importante: dentro de una línea de `crontab`, el carácter `%` tiene un significado especial (equivale a un salto de línea) — si quieres usarlo literalmente, como en `date +%V`, tienes que escaparlo como `\%`. Es un error de sintaxis muy fácil de cometer y que hace que la tarea no funcione como esperas, sin dar ningún aviso claro del motivo.

VAMOS A PRACTICAR: TRES TAREAS CON TEMPORALIDADES DISTINTAS

Tu profesor o profesora te va a facilitar un script `backup_personal.sh` que hace una copia de seguridad de tu carpeta personal. Vas a programar, junto a él, otros dos scripts que vas a escribir tú, cada uno con una temporalidad distinta.

Paso a paso:

1. Copia `backup_personal.sh` a tu carpeta personal y dale permisos de ejecución (`chmod +x`). Pruébalo una vez a mano para comprobar que funciona.
2. `crontab -e` y añade la línea de copia en semanas alternas (usa la que tienes más arriba, cambiando la ruta por la tuya y sin olvidar escapar el `%`).
3. Crea un script `saludo.sh` con este contenido (cambiando `tu_usuario` por el tuyo):

```
#!/bin/bash
echo "¡Hola $(whoami)! Son las $(date +%H:%M) del $(date +%A)." >>
/home/tu_usuario/saludo.log
```

4. Dale permisos de ejecución y prográmalo para que se ejecute **solo los días laborables** a una hora de tu elección: `0 9 * * 1-5 /home/tu_usuario/saludo.sh`.
5. Documenta, con capturas, las tres líneas de tu `crontab -l` completo (registro de prueba ya eliminado, backup en semanas alternas, saludo en días laborables).

Reflexiona: el crontab del ejercicio 2 se ejecuta técnicamente **todos** los domingos, no en semanas alternas. Entonces, ¿dónde está exactamente la lógica que hace que el backup real solo se ejecute la mitad de esas veces?

La tarea que necesita privilegios: el crontab de root

No todo lo que quieras automatizar lo puede hacer tu propio usuario. **Apagar el sistema**, por ejemplo, requiere privilegios de administrador — si programas esa tarea en tu propio `crontab -e`, fallará por falta de permisos. Las tareas que necesitan privilegios de administrador deben programarse en el **crontab de root**, que se edita con `sudo crontab -e` (un crontab completamente distinto al tuyo, no una versión "con sudo" del mismo).

VAMOS A PRACTICAR: PROGRAMA UN APAGADO AUTOMÁTICO

Paso a paso:

1. `sudo crontab -e` — abre el crontab de **root**, no el tuyo.
2. Para comprobar que el mecanismo funciona sin esperar de verdad a una hora concreta, añade primero una línea con una hora dentro de 1-2 minutos: `<minuto> <hora> * * * /usr/sbin/shutdown -h now` (sustituye por la hora exacta de dentro de un par de minutos). **Guarda cualquier trabajo abierto antes de este paso: la VM se va a apagar de verdad.**
3. Espera a que llegue la hora y comprueba que la VM se apaga sola. Vuelve a encenderla desde VirtualBox.
4. `sudo crontab -e` de nuevo, y cambia la hora a la definitiva: las 14:55 de todos los días (`55 14 * * * /usr/sbin/shutdown -h now`).
5. `sudo crontab -l` — documenta con una captura que la línea queda guardada en el crontab de root.

Reflexiona: si hubieras añadido esa misma línea en tu propio `crontab -e` (sin `sudo`) en vez de en el de root, ¿qué crees que habría pasado a la hora programada?

`at`: una sola vez, en un momento concreto

A diferencia de `cron`, **`at`** programa una tarea para que se ejecute **una única vez**, en un momento futuro que tú indicas: escribes `at 18:00`, introduces el comando (o comandos) y confirmas con Ctrl+D. `atq` lista las tareas pendientes; `atrm <id>` cancela una.

VAMOS A PRACTICAR: UNA TAREA PUNTUAL CON AT

Paso a paso:

1. `echo "echo Tarea completada > /home/tu_usuario/aviso.txt" | at now + 5 minutes` — programa la tarea (sustituye `tu_usuario` por tu usuario real).
2. `atq` — comprueba que la tarea aparece en la cola, con su hora prevista.
3. Espera los 5 minutos y ejecuta `cat /home/tu_usuario/aviso.txt` para comprobar que se ha ejecutado.

Reflexiona: si el script del ejercicio anterior hubiera usado `>> registro.log` en vez de la ruta absoluta, ¿dónde habría acabado escribiendo el archivo cuando lo ejecutara `cron`, y por qué no sería donde tú esperabas?

Caso práctico integrador

Trabajas en el departamento de soporte de una empresa. Un usuario con Ubuntu te avisa de que, tras una actualización del sistema, el equipo arranca en modo texto en vez de en modo gráfico. Además, te pide dos cosas: que un disco externo que usa para copias de seguridad se monte automáticamente cada vez que enciende el equipo, y que esa copia de seguridad se haga sola cada noche.

Con lo aprendido en esta unidad:

1. El sistema arranca en un target de texto en vez de gráfico. ¿Qué comando usarías para comprobar cuál es el target por defecto, y cuál para corregirlo?
2. Antes de tocar nada en la configuración del sistema, ¿qué medida preventiva deberías haber tomado primero?
3. Quieres que el disco externo se monte automáticamente en `/mnt/backup` en cada arranque. ¿Qué archivo editarías, y con qué dato identificarías el disco de forma fiable, aunque cambie el orden de detección de discos?
4. El usuario quiere la copia de seguridad automática cada noche a las 2:00. ¿Qué herramienta usarías, y qué precaución debes tener con las rutas dentro del script que la realice?
5. Si tras editar la configuración del disco el sistema no llega a arrancar con normalidad, ¿qué harías, paso a paso, para diagnosticarlo y solucionarlo?

Resumen de la unidad

- Tras el gestor de arranque, el kernel arranca systemd (PID 1), que organiza el sistema en unidades y targets (`multi-user.target` en texto, `graphical.target` con interfaz gráfica).
- Una sesión es el conjunto de procesos asociados a un usuario autenticado; cerrar sesión no es lo mismo que apagar el equipo.
- Un entorno de escritorio (GNOME, KDE Plasma, XFCE, MATE...) es la implementación concreta de la capa de shell gráfico; se pueden tener varios instalados a la vez y elegir cuál usar desde la pantalla de login.
- `/etc/fstab` define qué se monta automáticamente en cada arranque; usar UUID en vez del nombre de dispositivo evita problemas si cambia el orden de detección de discos.
- Un error en `fstab` puede impedir el arranque normal, pero es recuperable sin perder datos. En Ubuntu, con root bloqueado, se entra a corregirlo editando el arranque en GRUB (`init=/bin/bash`) o desde una ISO *live*. Tras editar `fstab`, `systemctl daemon-reload` y `mount -a` permiten detectar el error antes de reiniciar.
- `apt update` refresca el índice de paquetes disponibles; `apt upgrade` instala las actualizaciones — no son lo mismo.

- `remove` desinstala un paquete conservando su configuración; `purge` la elimina también. Un `.deb` suelto se instala con `dpkg -i`, resolviendo dependencias rotas con `apt --fix-broken install`.
- Snap es otro gestor de paquetes de Ubuntu, con paquetes autocontenidos; convive con APT sin conflicto, y tiene su propio `remove`/`remove --purge`.
- DNF y `.rpm` son el equivalente de APT y `.deb` en la familia Fedora/RHEL.
- NetworkManager (desde el entorno gráfico, `nmtui` o `nmcli`) y los asistentes de dispositivos simplifican configuraciones que también podrían hacerse a mano; en un Ubuntu de escritorio, es Netplan quien delega en NetworkManager, no al revés.
- `cron` programa tareas repetitivas —también con patrones como días laborables o semanas alternas— y necesita el crontab de root para tareas que requieren privilegios de administrador; `at`, tareas puntuales de una sola vez. Las rutas dentro de un script programado deben ser siempre absolutas.

Relación con RA y CE

CE	Contenido	Dónde se trabaja
a)	Arranque y parada del sistema, gestión de sesiones	Apartado 1
b)	Interfaces de usuario: línea de comandos frente a entorno gráfico	Apartado 2
c)	Preferencias del entorno personal (incluye varios entornos de escritorio)	Apartado 2
d)	Sistemas de archivos específicos: ext4 y `fstab`	Apartado 3
e)	Recuperación del sistema operativo	Apartado 3
f)	Actualización del sistema operativo	Apartado 4
g)	Instalación/desinstalación de utilidades	Apartado 4
h)	Asistentes de configuración: red y dispositivos	Apartado 5
i)	Automatización de tareas	Apartado 6